

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES CHENNAI
– 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCHEMA ANALYSIS AND VIVA VOCE

Course Code:	DSA05	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	Assessment:	Tool 2 – Schema Analysis and Viva Voce
Weightage:		Total Marks:	35 Marks

CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)

Student Name:	BHARANI DHARAN S	Reg. No.:	192424141
Section / Batch:	D/2024	Date of Submission:	06/08/26
Faculty Incharge:	Dr. B.SHYAMALA GOWRI	Project Mode:	Individual Submission

Code of Conduct

I BHARANI DHARAN S_ (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: BHARANI DHARAN S

Identification of Entities and Attributes (5 Marks)	Understanding of Relationships (5 Marks)	Analysis of Keys and Constraints (5 Marks)	Detection of Design Issues (10 Marks)	Viva Voce (5 Marks)	Total (35 Marks)

CO1 AT2 Application-Based Questions

Question 1

Employee Data Cleaning (CSV)

A company stores employee information in `employees.csv`. Some employee names contain extra spaces, salaries are stored as text, and a few records have missing values.

Task

- Read the CSV file.
- Remove extra spaces from employee names.
- Convert salary into numeric format.
- Fill missing salary values with the average salary.
- Save the cleaned dataset as `cleaned_employees.csv`.

Aim

To perform employee data cleaning using Pandas by reading a CSV file, removing unnecessary spaces from employee names, converting salary values into numeric format, filling missing salary values with the average salary, and exporting the cleaned dataset.

```
import pandas as pd
```

```
# Read the CSV file
```

```
df = pd.read_csv("employees.csv")
```

```
print("Original Dataset:")
```

```
print(df)
```

```
# Remove extra spaces from Name
```

```

df["Name"] = df["Name"].str.strip()

df["Name"] = df["Name"].str.replace(r"\s+", " ", regex=True)

# Convert Salary to numeric
df["Salary"] = pd.to_numeric(df["Salary"], errors="coerce")

# Fill missing salary with average salary
average_salary = df["Salary"].mean()

df["Salary"] = df["Salary"].fillna(average_salary)

print("\nAverage Salary:", average_salary)

print("\nCleaned Dataset:")

print(df)

# Save cleaned data
df.to_csv("cleaned_employees.csv", index=False)

print("\nFile saved successfully as 'cleaned_employees.csv'")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\Admin\Downloads\Employee_Project> &

Original Dataset:

	EmployeeID	Name	Department	Salary
0	E201	Rahul	HR	52000.0
1	E202	Priya	Finance	61000.0
2	E203	Kiran	IT	NaN
3	E204	Sneha	Marketing	58000.0
4	E205	Arjun	HR	56000.0
5	E206	Meera	Finance	NaN
6	E207	Vivek	IT	67000.0
7	E208	Ananya	Marketing	59000.0

Average Salary: 58833.33333333336

Cleaned Dataset:

	EmployeeID	Name	Department	Salary
0	E201	Rahul	HR	52000.000000
1	E202	Priya	Finance	61000.000000
2	E203	Kiran	IT	58833.333333
3	E204	Sneha	Marketing	58000.000000
4	E205	Arjun	HR	56000.000000
5	E206	Meera	Finance	58833.333333
6	E207	Vivek	IT	67000.000000
7	E208	Ananya	Marketing	59000.000000

Result

The employee dataset was successfully cleaned by removing extra spaces from employee names, converting salary values into numeric format, replacing missing salary values with the average salary, and saving the cleaned data as cleaned_employees.csv.

Bus Ticket Reservation System

Aim

To design and implement a **Bus Ticket Reservation System** database using SQLite by creating **Passenger**, **Bus**, and **Booking** tables and establishing relationships using primary and foreign keys.

PROGRAM

```
CREATE TABLE Passenger (  
    PassengerID INTEGER PRIMARY KEY,  
    Name TEXT NOT NULL  
);  
  
-- Create Bus Table  
CREATE TABLE Bus (  
    BusID INTEGER PRIMARY KEY,  
    Route TEXT NOT NULL  
);  
  
-- Create Booking Table  
CREATE TABLE Booking (  
    BookingID INTEGER PRIMARY KEY,  
    PassengerID INTEGER,  
    BusID INTEGER,  
    TravelDate DATE,  
    FOREIGN KEY (PassengerID) REFERENCES Passenger(PassengerID),  
    FOREIGN KEY (BusID) REFERENCES Bus(BusID)  
);  
  
-- Insert Sample Data  
INSERT INTO Passenger VALUES  
(1, 'Arun'),  
(2, 'Priya'),  
(3, 'Kavin');  
  
INSERT INTO Bus VALUES  
(101, 'Chennai - Coimbatore'),  
(102, 'Chennai - Madurai');  
  
INSERT INTO Booking VALUES
```

```
(1001, 1, 101, '2026-08-10'),  
(1002, 2, 102, '2026-08-11'),  
(1003, 1, 102, '2026-08-15');
```

-- Display Data

```
SELECT * FROM Passenger;
```

```
SELECT * FROM Bus;
```

```
SELECT * FROM Booking;
```

-- Show Passenger Booking Details

```
SELECT
```

```
    Passenger.Name,
```

```
    Bus.Route,
```

```
    Booking.TravelDate
```

```
FROM Booking
```

```
JOIN Passenger ON Booking.PassengerID = Passenger.PassengerID
```

```
JOIN Bus ON Booking.BusID = Bus.BusID;
```

PassengerID	Name
1	Arun
2	Priya
3	Kavin

SQL ▾

< 1 / 1 > 1 - 2 of 2



BusID	Route
101	Chennai - Coimbatore
102	Chennai - Madurai

SQL ▾

< 1 / 1 > 1 - 3 of 3



BookingID	PassengerID	BusID	TravelDate
1001	1	101	2026-08-10
1002	2	102	2026-08-11
1003	1	102	2026-08-15

SQL ▾

< 1 / 1 > 1 - 3 of 3



Name	Route	TravelDate
Arun	Chennai - Coimbatore	2026-08-10
Priya	Chennai - Madurai	2026-08-11
Arun	Chennai - Madurai	2026-08-15

Viva Questions & Answers

Q1. Why should booking details be stored separately from passenger information?

Answer:

Booking details are stored separately to avoid data duplication and maintain database normalization. A passenger can make multiple bookings for different buses and travel dates. Keeping booking information in a separate table improves data organization, reduces redundancy, and makes updates easier.

Q2. If a booking contains multiple passengers, how would you redesign the schema?

Answer:

A separate **BookingPassenger** table should be created.

Schema:

- **Passenger**(PassengerID, Name)
- **Bus**(BusID, Route)
- **Booking**(BookingID, BusID, TravelDate)
- **BookingPassenger**(BookingID, PassengerID)

This design allows one booking to include multiple passengers and supports a many-to-many relationship between bookings and passengers.

Result

The Bus Ticket Reservation System database was created successfully. The **Passenger**, **Bus**, and **Booking** tables were linked using primary and foreign keys. The system efficiently stores passenger details, bus information, and booking records while maintaining data integrity and minimizing data redundancy.